
N32WB031 VSCode GCC Environment Build Application Notes V1.0

Summary

This document mainly introduces the N32WB031 series Bluetooth chips, using the VSCode editing platform, ARM-GCC tool chain, to build the development and debugging environment.

CATALOGS

CATALOG

CATALOGS	2
INTRODUCTION.....	3
1 IMPORTING A KEIL/IAR PROJECT USING VSCODE	3
1.1 VSCODE EIED GCC TOOLCHAIN INSTALLATION	3
1.2 PROJECT IMPORT STEPS AND RELATED CONFIGURATIONS.....	3
1.3 PROJECT IMPORT	4
1.4 SELECTING THE GCC TOOLCHAIN.....	5
1.5 MODIFYING THE GCC STARTUP FILE	6
2 ENGINEERING COMPILATION PROBLEMS	7
2.1 PROBLEMS WITH THE USE OF THE KEYWORD __ALIGN.....	7
2.2 ASSEMBLY FUNCTION CALL PROBLEMS.....	7
2.3 PROBLEMS WITH THE USE OF THE KEYWORD __WEAK.....	9
2.3 UNDEFINED FUNCTION PROBLEMS	10
3 DOWNLOAD DEBUGGING	12
3.1 INSTALLING PACKAGES	12
3.2 ADDING CHIP MODELS TO JLINK.....	13
3.3 ADDING CHIP SUPPORT PACKAGES TO VSCODE.....	14
3.4 CODE DOWNLOAD	15
3.5 TEST RESULTS.....	16
REVISION HISTORY	18
LEGAL NOTICE.....	19

Introduction

The hardware platform of this document is based on N32WB031_STB_V1.3 development board, and the software development platform is based on VSCode + EIED + GCC Toolchain. although VSCode is only an editor, only the function of editing the code, but it has a wealth of plug-ins to support the development of MCUs and open source and free. Currently the more mainstream plug-ins are Cmake, EIED, Clion and so on. This application note uses EIDE (full name Embedded IDE). It is a plug-in for 8051/Cortex-M/RISC-V microcontroller development environment, can provide 8051/Cortex-M/RISC-V project development, compilation, burning and other functions on VSCode.

1 Importing a Keil/IAR project using VSCode

1.1 VSCode EIED GCC Toolchain Installation

VSCode, EIED and GCC toolchain installation process, please refer to the National Technology has been published "Vscode + GCC + GDB development environment build application notes V1.1.pdf", and application notes will not be described.

This subsection focuses on the following software and related plug-ins that need to be installed to build the N32WB031 VSCode+GCC environment:

- VSCode (official website download address: <https://code.visualstudio.com/Download>)
- Embedded IDE, C/C++, Cortex-Debug plug-ins for VSCode (just install them in VSCode Extensions, internet connection required).
- Jlink (recommended version V7.66 and below, download link: <https://www.segger.com/downloads/jlink/>)
- GCC toolchain
(Download link: <https://developer.arm.com/downloads/-/gnu-rm>, please note that you have to use cortex-r/m version, the author is currently using the latest version: gcc-arm-none-eabi-10.3-2021.10-win32)

1.2 Project import steps and related configurations

Importing an existing Keil/IAR project for WB031 using VSCode is divided into the following steps:

- Project Import
- Selecting the GCC Toolchain
- Modify the gcc startup file

1.3 Project import

One of the benefits of using EIDE is that it can directly import keil/IAR project, the project structure of keil/IAR can be directly imported into VSCode, users do not need to rebuild the project, so as to achieve a seamless connection. Open the installed Vscode, click on the left side of the chip icon of the EIED plug-in, the EIED PROJECTS project management interface will pop up, here only need to import the ready-made keil/IAR project, as shown in Figure 1, select Import Project, import the rdtss project used in this document, as shown in Figure 2.

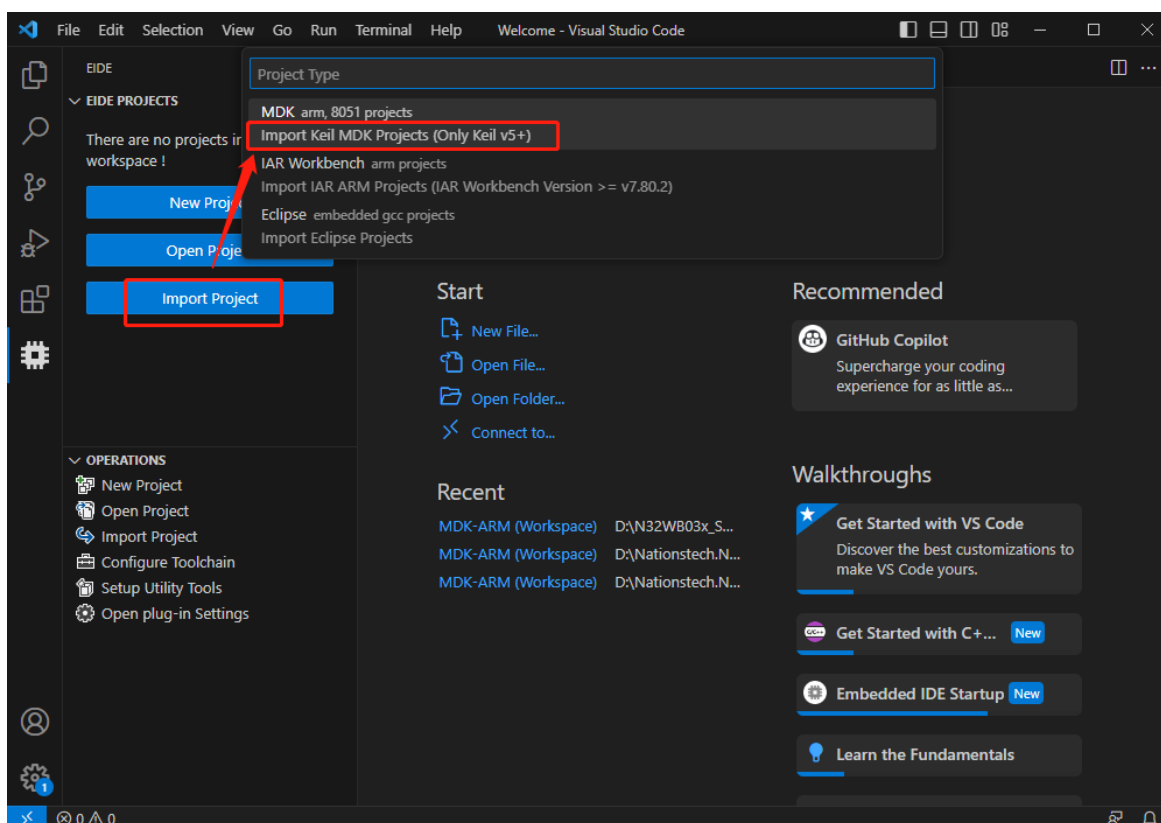


Figure 1 VSCode imported into keil project

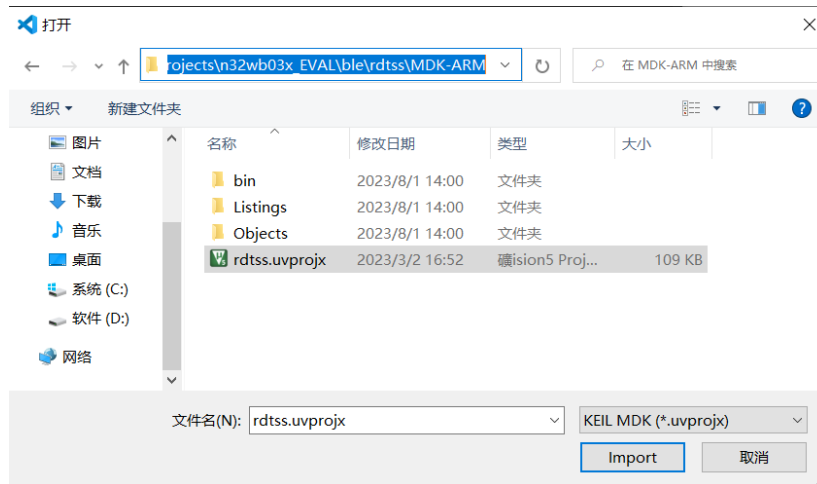


Figure 2 VSCode importing rdtss project

1.4 Selecting the GCC Toolchain

This routine uses the GCC toolchain to compile the project, so the toolchain configuration items to select GCC. EIED plug-in default is to use AC5, as shown in Figure 3, select "Builder Configurations: AC5", on the right side will pop up AC5, AC6, GCC, IAR_ARM, select GCC, you can see the toolchain Builder Configurations: GCC.

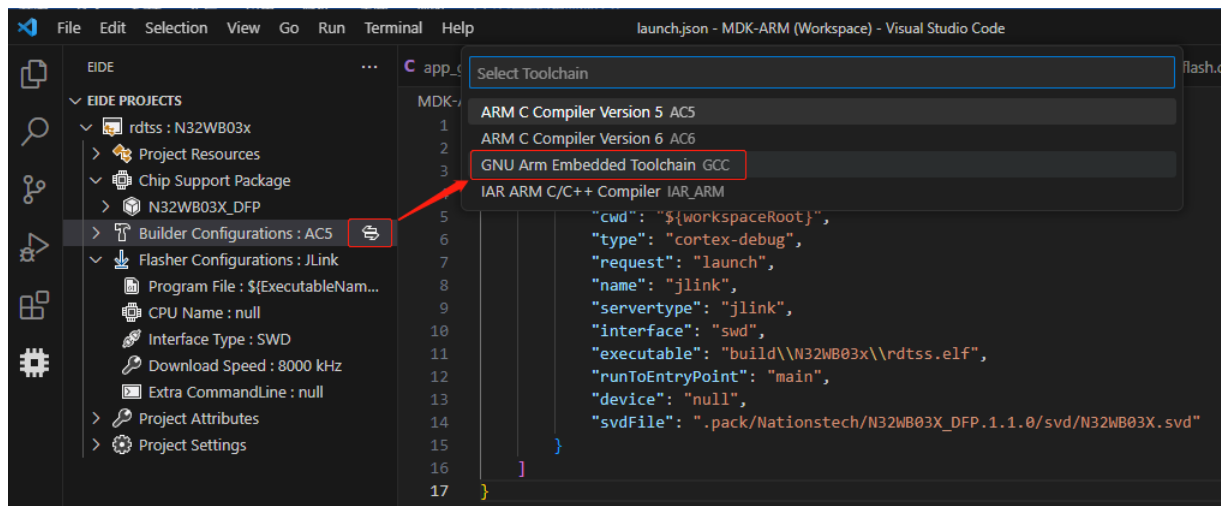


Figure 3 Selecting the GNU toolchain

Under the Builder Configurations item, there is also Linker Script File Path, this is the linker script path configuration, the linker script has been put into the N32WB03x_SDK_V1.3/firmware\CMSIS\device\n32wb03x_flash.ld, users can choose according to their own storage project path. Under the path, the user according to their own storage project path to choose can be. As shown in Figure 4, remember to hit the Enter key after entering the path. Link script selection above a CPU Tyte option, WB031 is based on Cortex-M0 core, select Cortex-M0 can be.

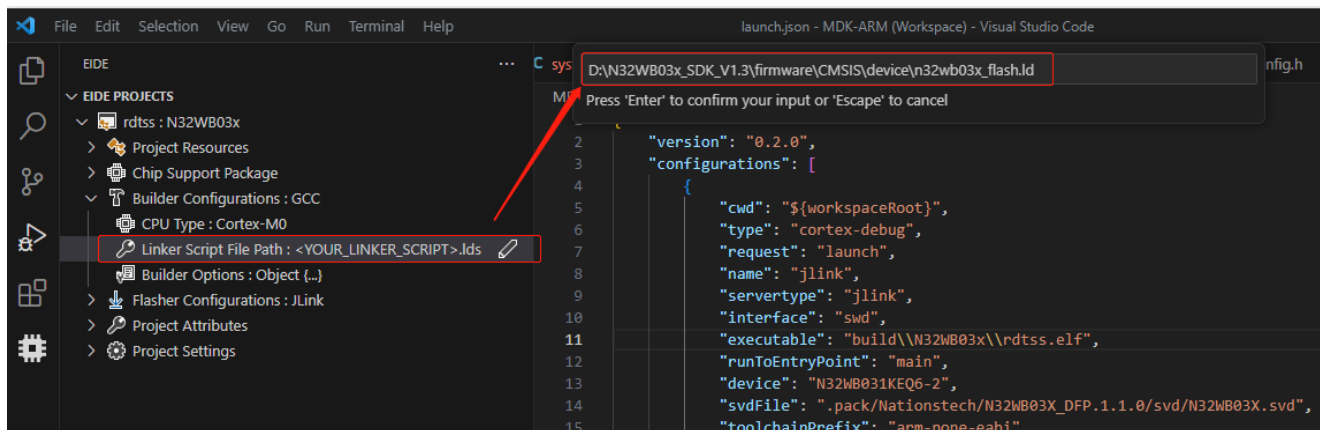


Figure 4 Importing linked scripts

1.5 Modifying the GCC Startup File

Since the startup file in keil/IAR has its own special format and syntax, which is not compatible with GCC assembler, it is necessary to provide a separate startup file written in GUN assembly syntax to the GCC project. This startup file has been placed in N32WB03x_SDK_V1.3\firmware\

Under the path of CMSIS\device\startup, just delete the original startup_n32wb03x.s startup file from the project and re-add startup_n32wb03x_gcc.s to the project, as shown in Figure 5.

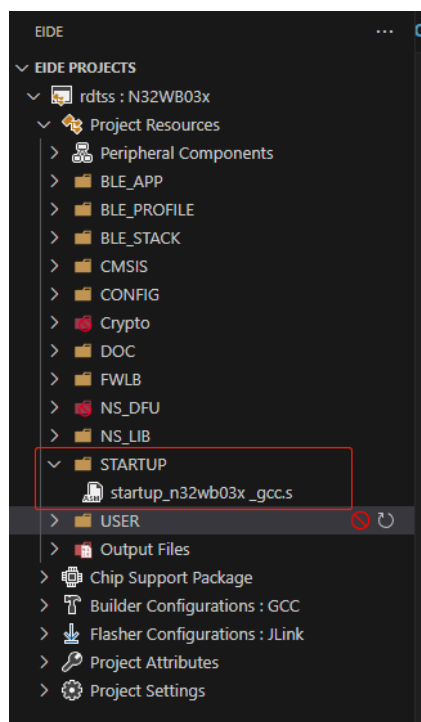


Figure 5 Replacing the gcc startup file

2 Engineering compilation problems

In accordance with the above sections to import, configure the project can be compiled after the project, due to ARMCC and GCC compiler in the compilation of some syntax is not the same, the compilation process inevitably has some syntax errors, the following description of the import of Keil project compilation errors in the process.

2.1 Problems with the use of the keyword `__align`

Press the shortcut key F7 or click the Compile button in the upper right corner of the software interface to compile the project. As shown in Figure 6 below, there is a compilation error in the log of the compilation result:

n32wb03x_qflash.c:87:16: error: expected declaration specifiers or '...' before numeric constant.

```

PROBLEMS 14 OUTPUT DEBUG CONSOLE TERMINAL MEMORY XRTOS
.../.../.../middlewares/Nationtech/ble_library/ns_ble_stack/arch/arch.h:322:1: warning: multi-line comment [-Wcomment]
322 | // do {
    | ^
>> [ 7%] CC '.../.../firmware/n32wb03x_std_periph_driver/src/n32wb03x_qflash.c'
.../.../.../firmware/n32wb03x_std_periph_driver/src/n32wb03x_qflash.c:87:16: error: expected declaration specifiers or '...' before numeric constant
87 | static __align(4) unsigned char _acCMD_BIN[0x324] = {
    | ^
.../.../.../firmware/n32wb03x_std_periph_driver/src/n32wb03x_qflash.c: In function 'Qflash_Init':
.../.../.../firmware/n32wb03x_std_periph_driver/src/n32wb03x_qflash.c:169:39: error: '_acCMD_BIN' undeclared (first use in this function)
169 | CMD_FLASH_SE = (CMD_FLASH_SE_t *)(_acCMD_BIN + 0xA1);
    | ^
.../.../.../firmware/n32wb03x_std_periph_driver/src/n32wb03x_qflash.c:169:39: note: each undeclared identifier is reported only once for each function object
>> [ 10%] CC '.../.../.../middlewares/Nationtech/ble_library/ns_ble_profile/bas/bass/src/bass.c'

```

Figure 6 Problems with the use of the GCC compilation keyword `__align`

From the tip of the n32wb03x_qflash.c file line 87 expects to specify the declaration, this problem is due to keil in the alignment keyword `__align` usage and GUN environment using `align` is not the same as that led to, such as the keil project to specify a certain array to 4 bytes aligned, directly written as `__align(4) array[]` can be, while in the GNU environment need to be written in the form of `__attribute__((aligned(4))) array[]`, the definition of the array in line 87 of n32wb03x_qflash.c is written in the following format:

```
static __attribute__((aligned(4))) unsigned char _acCMD_BIN[0x324]
```

Recompile and the error message in Figure 6 is gone.

Similarly the array definition on line 60 in ns_ble.c needs to be changed to the following format:

```
static __attribute__((aligned(4))) unsigned char rwip_fifo_isr_codeArray[]
```

2.2 Assembly function call problems

After recompiling after resolving the above byte alignment format issue, there will be an error message in the log displaying the results as shown in Figure 7 below.

Figure 7 Problems with C calling assembly functions under GCC

```
.text
.syntax unified
.thumb

/* Complete site protection prior to cross-media jump calls */

.type system_delay_cycles, %function //declare this as a function
.global system_delay_cycles //Export symbols to make them visible to the outside world, i.e., you can export functions
or global variables.

system_delay_cycles.
subs r0, #1
bne system_delay_cycles
bx lr

.end
```

Create a new .s file named system_delay.s, add this file to the CMSIS directory in the project, as shown in Figure 8, and then comment out the system_delay_cycles function in sytem_n32wb03x.c.

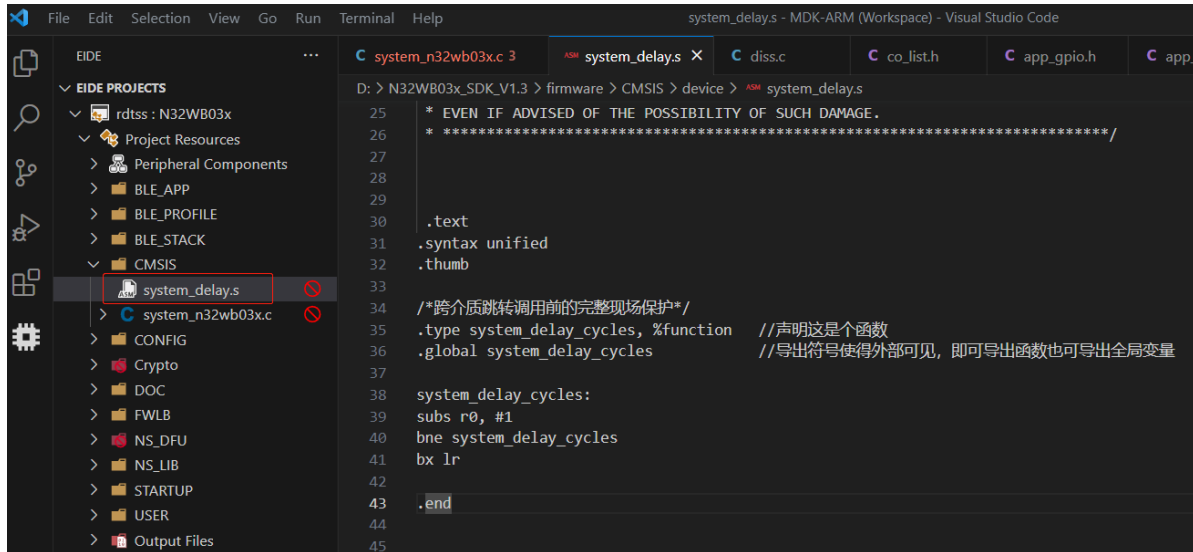


Figure 8 Adding the system_delay.s file

2.3 Problems with the use of the keyword __weak

After solving the above assembly function call problem recompile, will compile the log area will have an error message as shown in Figure 9. From the error message can only see the wrong point, can not see the cause of the problem. Open the tip of the ns_sleep.c file, locate the 204th line __weak void app_sleep_prepare __proc(void). This is a weak definition of the app_sleep_prepare_proc function. With the __weak modifier, the user can redefine a function with the same name in the user's file, and the compiler will choose the user-defined function for compilation. The compiler will not report an error. In keil project, it is enough to add __weak directly in front of the function name, but in GNU, you need to use __attribute__((weak)) to define the function weakly to output its symbol. Therefore it should be written in the following format:

```
__attribute__((weak)) void app_sleep_prepare_proc(void).
```

Similarly, the app_sleep_resume_proc function on line 216 of the ns_sleep.c file is modified in this manner.

```

198 /**
199  * @brief User code beofre enter sleep mode
200  * @param
201  * @return
202  * @note
203  */
204 __weak void app_sleep_prepare_proc(void)
205 {
206
207 }
208
209 /**
210  * @brief User code after out of sleep mode. This function run after interrupt
211  * handler function if any interrupt pending when sleep.
212  * @param

```

PROBLEMS 11 OUTPUT DEBUG CONSOLE TERMINAL MEMORY XRTOS

```

314 | // do {
.../.../.../middlewares/Nationtech/ble_library/ns_ble_stack/arch/arch.h:322:1: warning: multi-line comment [-Wcomment]
322 | // do {
.../.../.../middlewares/Nationtech/ble_library/ns_library/sleep/ns_sleep.c:204:7: error: expected ';' before 'void'
204 | __weak void app_sleep_prepare_proc(void)
.../.../.../middlewares/Nationtech/ble_library/ns_library/sleep/ns_sleep.c:216:7: error: expected ';' before 'void'
216 | __weak void app_sleep_resume_proc(void)

```

Figure 9 __weak usage tips

2.3 Undefined Function Problems

When the above weak function definition issues are resolved, recompile and there are still a bunch of UNDEFINED REFERENCE issues, as shown in Figure 10:

PROBLEMS 51 OUTPUT DEBUG CONSOLE TERMINAL MEMORY XRTOS

```

Nationtech/ble_library/ns_ble_profile/dis/diss/src/diss.o: in function `diss_destroy':
d:\W32MB03x_SDK_V1.3\projects\n32wb03x_EVAL\ble\rdtss\MDK-ARM\.../.../.../middlewares/Nationtech/ble_library/ns_ble_profile/dis/diss/src
undefined reference to `co_list_pop_front'
c:/users/nationz/.eide/tools/gcc_arm/bin/./lib/gcc/arm-none-eabi/10.2.1/.../arm-none-eabi/bin/ld.exe: d:\W32MB03x_SDK_V1.3\projects\n
ble\rdtss\MDK-ARM\.../.../.../middlewares/Nationtech/ble_library/ns_ble_profile/dis/diss/src/diss.c:194: undefined reference to `ke_free'
c:/users/nationz/.eide/tools/gcc_arm/bin/./lib/gcc/arm-none-eabi/10.2.1/.../arm-none-eabi/bin/ld.exe: d:\W32MB03x_SDK_V1.3\projects\n
ble\rdtss\MDK-ARM\.../.../.../middlewares/Nationtech/ble_library/ns_ble_profile/dis/diss/src/diss.c:199: undefined reference to `ke_free'
c:/users/nationz/.eide/tools/gcc_arm/bin/./lib/gcc/arm-none-eabi/10.2.1/.../arm-none-eabi/bin/ld.exe: ./build/N32MB03x/_/_/_/_/_/_
Nationtech/ble_library/ns_ble_profile/dis/diss/src/diss.o: in function `diss_init':
d:\W32MB03x_SDK_V1.3\projects\n32wb03x_EVAL\ble\rdtss\MDK-ARM\.../.../.../middlewares/Nationtech/ble_library/ns_ble_profile/dis/diss/src
undefined reference to `atm_svc_create_db'
c:/users/nationz/.eide/tools/gcc_arm/bin/./lib/gcc/arm-none-eabi/10.2.1/.../arm-none-eabi/bin/ld.exe: d:\W32MB03x_SDK_V1.3\projects\n
ble\rdtss\MDK-ARM\.../.../.../middlewares/Nationtech/ble_library/ns_ble_profile/dis/diss/src/diss.c:154: undefined reference to `ke_malloc'
c:/users/nationz/.eide/tools/gcc_arm/bin/./lib/gcc/arm-none-eabi/10.2.1/.../arm-none-eabi/bin/ld.exe: d:\W32MB03x_SDK_V1.3\projects\n
ble\rdtss\MDK-ARM\.../.../.../middlewares/Nationtech/ble_library/ns_ble_profile/dis/diss/src/diss.c:167: undefined reference to `co_list'
c:/users/nationz/.eide/tools/gcc_arm/bin/./lib/gcc/arm-none-eabi/10.2.1/.../arm-none-eabi/bin/ld.exe: d:\W32MB03x_SDK_V1.3\projects\n
ble\rdtss\MDK-ARM\.../.../.../middlewares/Nationtech/ble_library/ns_ble_profile/dis/diss/src/diss.c:170: undefined reference to `ke_stat'
c:/users/nationz/.eide/tools/gcc_arm/bin/./lib/gcc/arm-none-eabi/10.2.1/.../arm-none-eabi/bin/ld.exe: ./build/N32MB03x/_/_/_/_/_/_
Nationtech/ble_library/ns_ble_profile/dis/diss/src/diss.o: in function `diss_value_cfm handler':

```

Figure 10 Function Undefined Tip

This is because these functions are Bluetooth library functions, have been encapsulated masked to the ROM, can only be called through the API function interface, where the compiler reported an error because the compiler in the compilation, linking time can not find these functions linked to the entry address. The reason why the keil project can be compiled and linked normally is that when building the wb031 keil project, under the Options for Target tab, the Linker/Keil project can be compiled and linked normally.

The Misc controls column imports the function symbol lists for these library functions into the project, and when compiling and linking the compiler automatically looks for external function symbols under the paths specified in the tab. As shown in Figure 11:

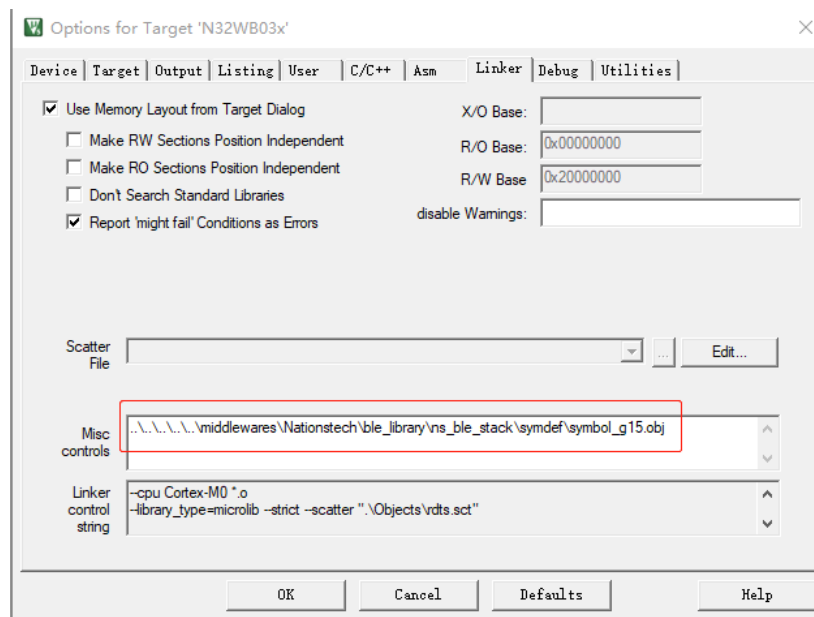


Figure 11 Library function symbol table path configuration under keil

About function symbols are not explained here, interested readers are recommended to refer to the following article:

<https://blog.csdn.net/ai5945fei/article/details/115897944>

(Note: The external function symbol table in this routine is located in

N32WB03x_SDK_V1.3\middlewares\Nationstech\ble_library\ns_ble_stack\symdef\symbol_g15.obj, a semicolon in the file means to comment out this line)

Under GCC, it is also necessary to import this external symbol table into the project. However, the format of external symbols under GNU is different from Keil. GCC does not recognize the .obj format, but it can recognize the .txt format. Therefore, the need for this symbol table for the next format conversion, here has been a good conversion, the reader only need to import directly. Import steps as shown in Figure 12, please note that the path of the file in the parent directory and subdirectory separator with a double slash "\\".

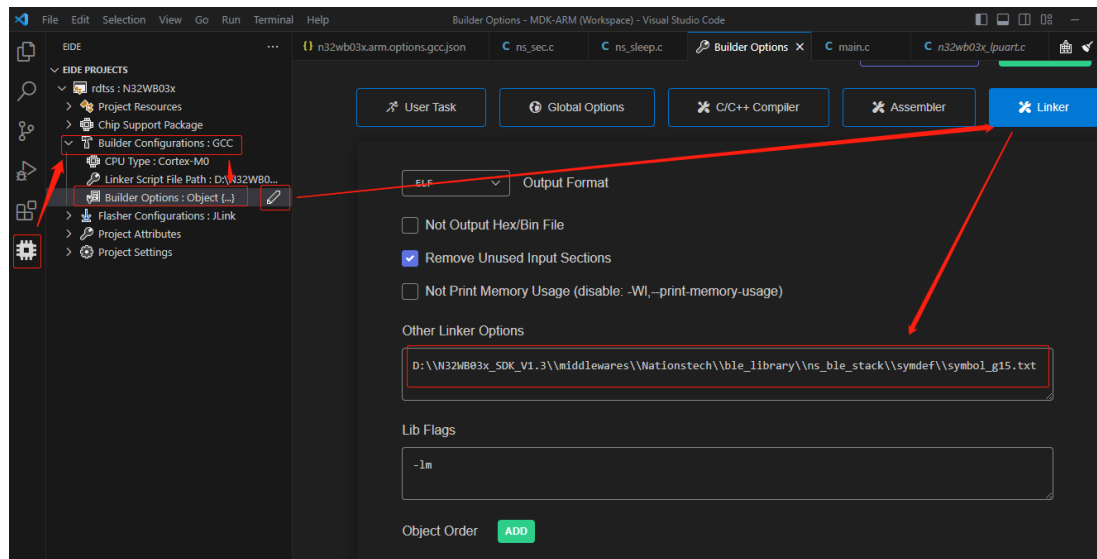


Figure 12 Symbol table for importing external functions under VSCode

After installing the above configured external symbol table, click "SAVE ALL" to save the configuration, and then compile the project again, you will eventually get a successful compilation of the prompt as shown in Figure 13.

```
[ INFO ] start linking ...

Memory region      Used Size  Region Size  %age Used
FLASH:             36624 B    256 KB      13.97%
RAM:               6864 B     32 KB       20.95%

[ INFO ] start outputting files ...

>> output hex file      [done]

file path: "build/N32WB03x/rdtss.hex"

>> output bin file      [done]

file path: "build/N32WB03x/rdtss.bin"

[ DONE ] build successfully !, elapsed time 0:0:7

* Terminal will be reused by tasks, press any key to close it.
```

Figure 13 Compile Success Tip

3 Download Debugging

3.1 Installing packages

Installation of N32WB031 Development Kit N32WB03x_V1.3.0\5-Software Development Kit path under the N32WB03x_DFP.1.1.0.packages, the installation path user-selected, I chose the default path, the final installation results as shown in Figure 14, this path to be used later.

« 用户 > nationz > AppData > Local > Arm > Packs > .Download				在 .Dow
名称	修改日期	类型	大小	
ARM.CMSIS.5.7.0.pack	2020/4/9 15:06	uVision Software P...	113,518 KB	
ARM.CMSIS.5.7.0.pdsc	2020/4/9 15:47	PDSC 文件	268 KB	
ARM.CMSIS-Driver.2.6.1.lic.txt	2018/8/14 10:26	文本文档	12 KB	
ARM.CMSIS-Driver.2.6.1.pack	2020/9/21 18:49	uVision Software P...	577 KB	
ARM.CMSIS-Driver.2.6.1.pdsc	2020/7/13 10:50	PDSC 文件	25 KB	
Keil.ARM_Compiler.1.6.3.pack	2020/4/22 7:32	uVision Software P...	3,972 KB	
Keil.ARM_Compiler.1.6.3.pdsc	2020/4/22 8:31	PDSC 文件	20 KB	
Keil.MDK-Middleware.7.12.0.lic.rtf	2020/4/29 16:17	RTF 格式	222 KB	
Keil.MDK-Middleware.7.12.0.pack	2020/7/2 18:46	uVision Software P...	110,901 KB	
Keil.MDK-Middleware.7.12.0.pdsc	2020/7/1 15:35	PDSC 文件	266 KB	
Nationtech.N32G45x_DFP.1.0.5.pdsc	2023/7/28 10:35	PDSC 文件	60 KB	
Nationtech.N32G45x_DFP.1.0.6.pack	2023/7/13 14:45	uVision Software P...	143 KB	
Nationtech.N32G45x_DFP.1.0.6.pdsc	2023/7/13 11:12	PDSC 文件	69 KB	
Nationtech.N32WB03X_DFP.1.1.0.pack	2022/5/13 14:51	uVision Software P...	217 KB	
Nationtech.N32WB03X_DFP.1.1.0.pdsc	2022/1/4 17:06	PDSC 文件	16 KB	

Figure 14 Pack Pack Installation Results

3.2 Adding Chip Models to Jlink

Copy the following contents to the end of C:\Program Files\SEGGER\Jlink\JLinkDevices.xml in the installation directory of jlink (Jlink version: V7.66), as shown in Figure 15.

```
<Device>
  <ChipInfo Vendor="Nationtech" Name="N32WB031KEQ6-2" Core="JLINK_CORE_CORTEX_M0"
WorkRAMAddr="0x20000000" WorkRAMSize="0x0000C000" /& gt;
  <FlashBankInfo Name="External Flash" BaseAddr="0x01000000" MaxSize="0x00040000" Loader=
"Devices\Nationtech\N32WB03X.FLM" LoaderType="FLASH_ALGO_TYPE_OPEN" AlwaysPresent="1"/>
</Device>
```

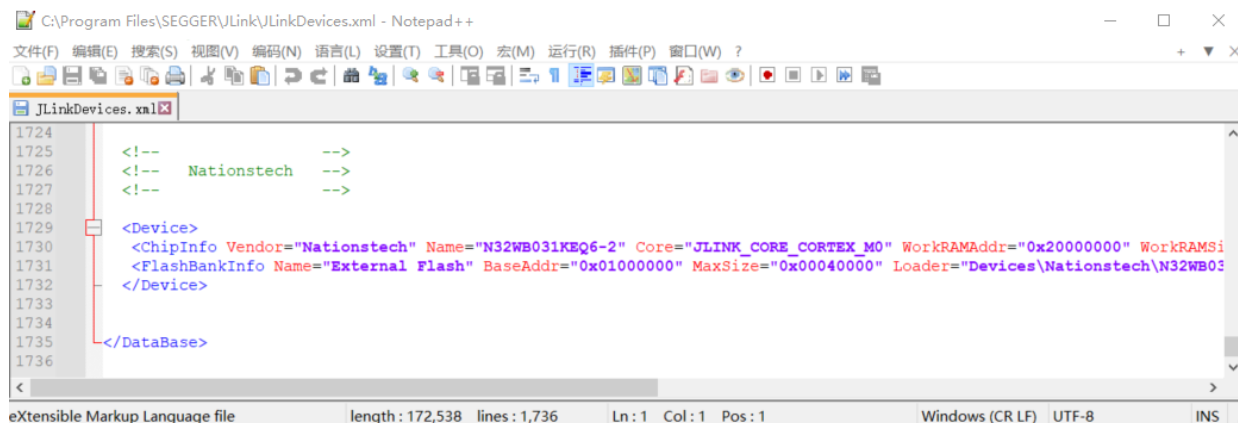


Figure 15 Adding WB031KEQ6-2 in jlinkDevices.xml

Create a folder Nationtech under the Jlink installation directory C:\Program Files\SEGGER\JLink\Devices and copy the N32WB03x.FLM file to this folder as shown in Figure 16.

系统 (C:) > Program Files > SEGGER > JLink > Devices > Nationtech				
名称	修改日期	类型	大小	
N32G032x_32.FLM	2021/6/9 10:12	FLM 文件	12 KB	
N32G032x_64.FLM	2021/6/9 10:12	FLM 文件	12 KB	
N32G43x_64.FLM	2021/6/9 10:12	FLM 文件	12 KB	
N32G43x_128.FLM	2021/6/9 10:12	FLM 文件	12 KB	
N32G45x_NRP_128K.FLM	2021/6/9 10:12	FLM 文件	13 KB	
N32G45x_NRP_256K.FLM	2021/6/9 10:12	FLM 文件	13 KB	
N32G45x_NRP_512K.FLM	2021/6/9 10:12	FLM 文件	12 KB	
N32L40x_64.FLM	2021/6/9 10:12	FLM 文件	12 KB	
N32L40x_128.FLM	2021/6/9 10:12	FLM 文件	12 KB	
N32L43x_64.FLM	2021/6/9 10:12	FLM 文件	12 KB	
N32L43x_128.FLM	2021/6/9 10:12	FLM 文件	12 KB	
N32WB03x.FLM	2022/1/4 16:39	FLM 文件	28 KB	

Figure 16 Copy N32WB03x.FLM

3.3 Adding Chip Support Packages to VSCode

Select EIED->Chip Support Package->From Disk in order, choose the installation path of the PACK package in chapter 4.1, and import the pack package. As shown in Figure 17, 18, then import the chip model. Only after importing the chip support package, you can view the value of the chip registers during the subsequent development and debugging.

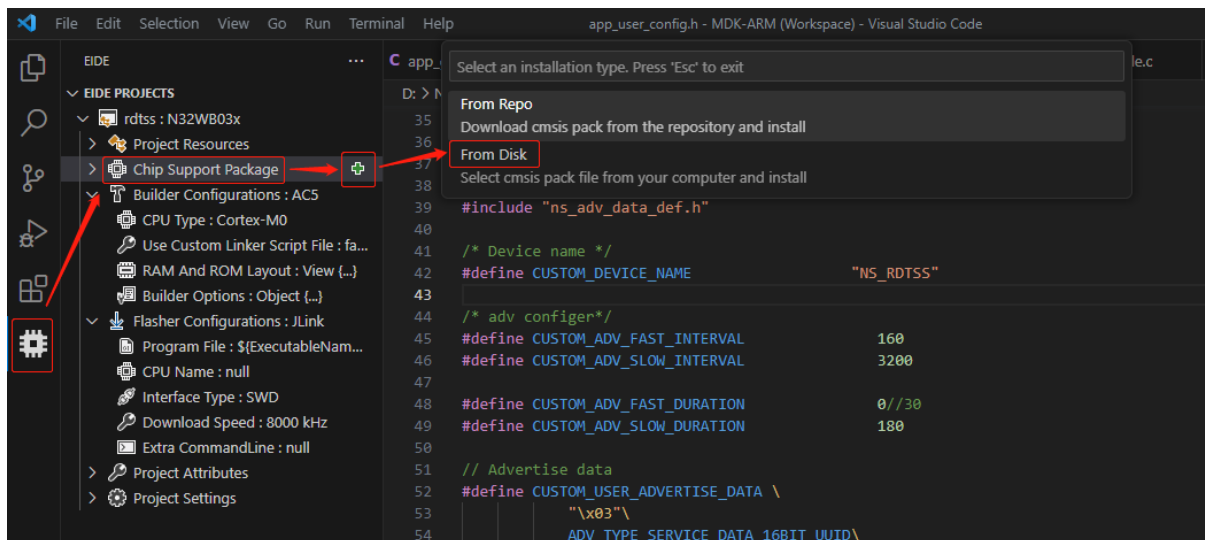


Figure 17 Selecting a Chip Support Package

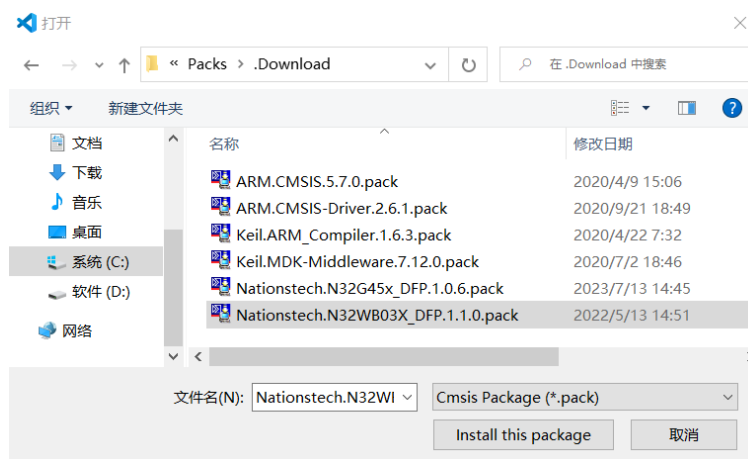


Figure 18 Importing the N32WB03X pack

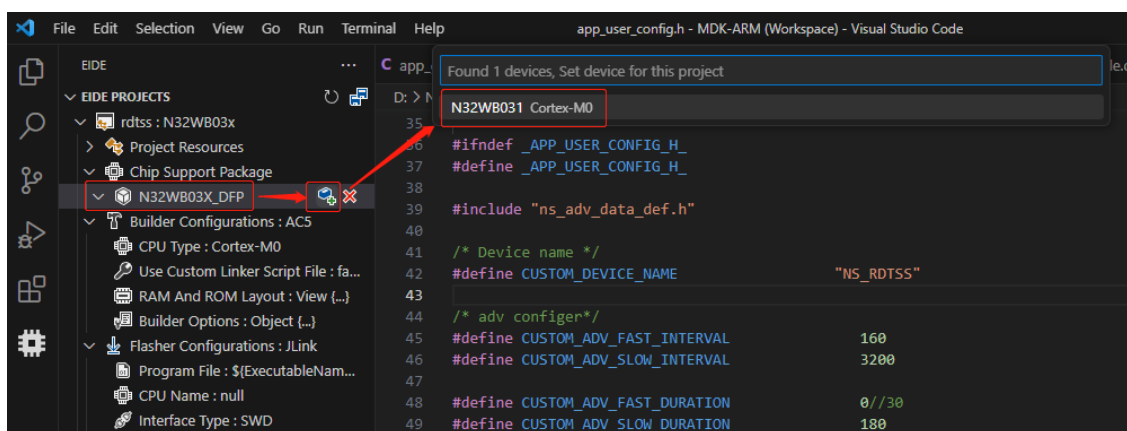


Figure 19 Importing Chip Models

3.4 Code Download

Ensure that the above steps are completed, then you can proceed with the download normally. Click the Download button in the upper right corner of VSCode, and the download interface will pop up as shown in Figure 20:

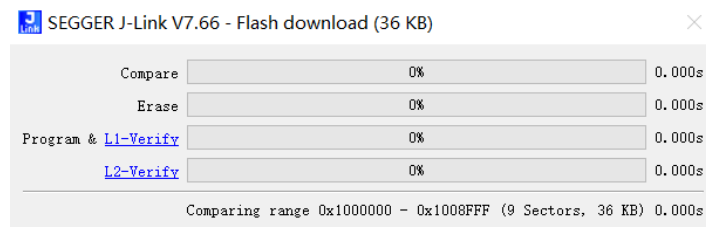


Figure 20 Jlink Download Interface

```

PROBLEMS 167 OUTPUT DEBUG CONSOLE TERMINAL MEMORY XRTOS

CPUTID register: 0x410CC200. Implementer code: 0x41 (ARM)
Found Cortex-M0 r0p0, Little endian.
FPUnit: 4 code (BP) slots and 0 literal slots
CoreSight components:
ROMtbl[0] @ E00FF000
[0][0]: E000E000 CID B105E000 PID 000BB000 SCS
[0][1]: E0001000 CID B105E000 PID 000BB00A DWT
[0][2]: E0002000 CID B105E000 PID 000BB00B FPB
Cortex-M0 identified.
Reset delay: 0 ms
Reset type NORMAL: Resets core & peripherals via SYSRESETREQ & VECTRESET bit.
Reset: Halt core after reset via DEMCR.VC_CORERESET.
Reset: Reset device via AIRCR.SYSRESETREQ.
J-Link>halt
PC = 00000100, CycleCnt = 00000000
R0 = FFFFFFFF, R1 = FFFFFFFF, R2 = FFFFFFFF, R3 = FFFFFFFF
R4 = FFFFFFFF, R5 = FFFFFFFF, R6 = FFFFFFFF, R7 = FFFFFFFF
R8 = FFFFFFFF, R9 = FFFFFFFF, R10 = FFFFFFFF, R11 = FFFFFFFF
R12 = FFFFFFFF
SP(R13) = 2000C000, MSP = 2000C000, PSP = FFFFFFFC, R14(LR) = FFFFFFFF
XPSR = C1000000: APSR = NZcvq, EPSR = 01000000, IPSR = 000 (NoException)
CFBP = 00000000, CONTROL = 00, FAULTMASK = 00, BASEPRI = 00, PRIMASK = 00
FPU regs: FPU not enabled / not implemented on connected CPU.
J-Link>loadfile "d:\N32WB03x_SDK_V1.3\projects\n32wb03x_EVAL\ble\rdtss\MDK-ARM\build\N32WB03x\rdtss.hex"
Downloading file [d:\N32WB03x_SDK_V1.3\projects\n32wb03x_EVAL\ble\rdtss\MDK-ARM\build\N32WB03x\rdtss.hex]...
J-Link: Flash download: Bank 0 @ 0x01000000: Skipped. Contents already match
O.K.
J-Link>r
Reset delay: 0 ms
Reset type NORMAL: Resets core & peripherals via SYSRESETREQ & VECTRESET bit.
Reset: Halt core after reset via DEMCR.VC_CORERESET.
Reset: Reset device via AIRCR.SYSRESETREQ.
J-Link>go
J-Link>exit

```

Figure 21 Finish Download Screen

3.5 Test results

```

D: > N32WB03x_SDK_V1.3 > projects > n32wb03x_EVAL > ble > rdtss > inc > C app_user_config.h > ...
28  /*
29  * @file app_user_config.h
30  * @author Nations Firmware Team
31  * @version v1.0.1
32  *
33  * @copyright Copyright (c) 2019, Nations Technologies Inc. All rights reserved.
34  */
35
36 #ifndef _APP_USER_CONFIG_H
37 #define _APP_USER_CONFIG_H
38
39 #include "ns_adv_data_def.h"
40
41 /* Device name */
42 #define CUSTOM_DEVICE_NAME "NS_RDTSS_20230802"
43
44 /* adv configer*/
45 #define CUSTOM_ADV_FAST_INTERVAL 160
46 #define CUSTOM_ADV_SLOW_INTERVAL 3200
47
48 #define CUSTOM_ADV_FAST_DURATION 0//30
49 #define CUSTOM_ADV_SLOW_DURATION 180

```

Figure 22

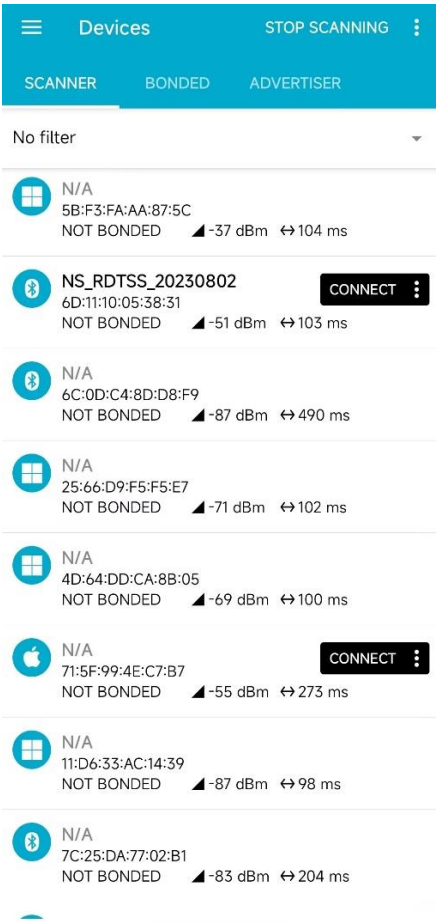


Figure 23

Revision History

releases	dates	note
V1.0	2023.08.02	New Document

Legal Notice

This document is the exclusive property of Nations Technologies Inc. (Hereinafter referred to as NATIONS). This document, and the product of NATIONS described herein (Hereinafter referred to as the Product) are owned by NATIONS under the laws and treaties of the People's Republic of China and other applicable jurisdictions worldwide.

NATIONS does not grant any license under its patents, copyrights, trademarks, or other intellectual property rights. Names and brands of third party may be mentioned or referred thereto (if any) for identification purposes only.

NATIONS reserves the right to make changes, corrections, enhancements, modifications, and improvements to this document at any time without notice. Please contact NATIONS and obtain the latest version of this document before placing orders.

Although NATIONS has attempted to provide accurate and reliable information, NATIONS assumes no responsibility for the accuracy and reliability of this document.

It is the responsibility of the user of this document to properly design, program, and test the functionality and safety of any application made of this information and any resulting product. In no event shall NATIONS be liable for any direct, indirect, incidental, special, exemplary, or consequential damages arising in any way out of the use of this document or the Product.

NATIONS Products are neither intended nor warranted for usage in systems or equipment, any malfunction or failure of which may cause loss of human life, bodily injury or severe property damage. Such applications are deemed, "Insecure Usage".

Insecure usage includes, but is not limited to: equipment for surgical implementation, atomic energy control instruments, airplane or spaceship instruments, all types of safety devices, and other applications intended to support or sustain life.

All Insecure Usage shall be made at user's risk. User shall indemnify NATIONS and hold NATIONS harmless from and against all claims, costs, damages, and other liabilities, arising from or related to any customer's Insecure Usage.

Any express or implied warranty with regard to this document or the Product, including, but not limited to, the warranties of merchantability, fitness for a particular purpose and non-infringement are disclaimed to the fullest extent permitted by law.

Unless otherwise explicitly permitted by NATIONS, anyone may not use, duplicate, modify, transcribe or otherwise distribute this document for any purposes, in whole or in part.